



PLUGIN OPEN SOURCE PARA O CLAUDE CODE

Obsidian Memory

Memória persistente para cada projeto. O Claude retoma de onde parou, sem você explicar de novo e sem reler o código.

Markdown puro

Git e Obsidian opcionais

macOS · Linux · Windows

Open source · MIT

Cada sessão do agente começa do zero

RETOMAR CUSTA CARO

Você explica tudo de novo

O que é o projeto, onde parou, o que já foi decidido. Depois o agente relê README, código e histórico do git para se orientar.

COMPACTAÇÃO

O contexto encolhe e some

Em sessões longas, a conversa é resumida. Detalhes, motivos e próximos passos se perdem no resumo.

DECISÕES REABERTAS

O "porquê" não está no código

O código mostra o que foi feito, não por quê. O agente propõe de novo o que já foi descartado.

TOKENS E TEMPO

Tokens gastos só para se situar

Cada leitura de arquivo é mais uma chamada ao modelo, que reprocessa todo o contexto. Isso acontece antes de começar o trabalho de verdade.

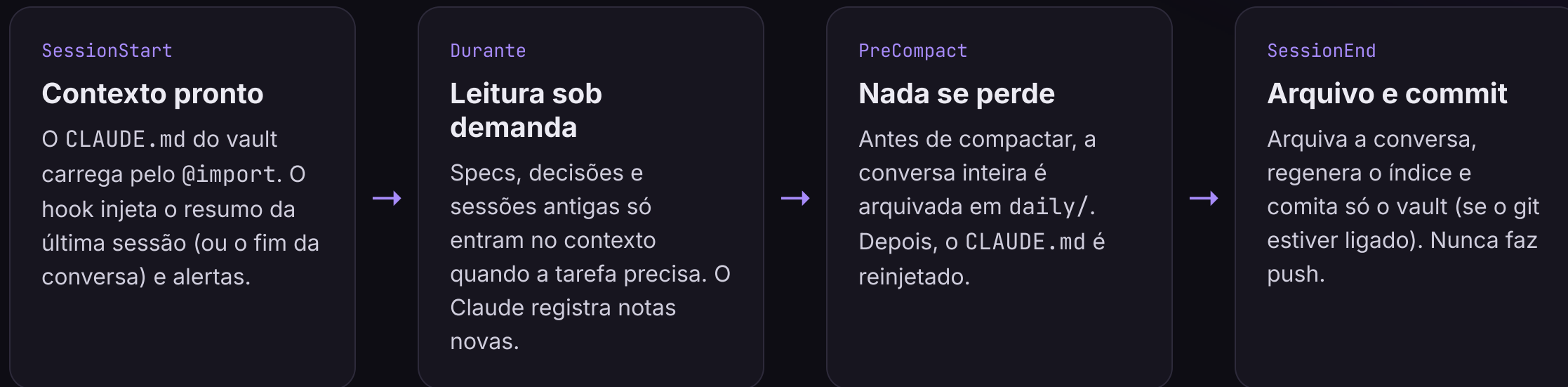
Um vault de memória dentro de cada projeto

A pasta `obsidian-vault/` fica junto do código. É Markdown puro: o Claude lê e escreve, e você pode abrir no Obsidian se quiser.

- ◆ **CLAUDE.md de contexto:** projeto, stack, estado atual, decisões, mapa de arquivos e o que não fazer
- ◆ **Histórico por dia:** resumo de cada sessão e a conversa arquivada
- ◆ **Notas do projeto:** specs, planos, processos, decisões (ADRs), bugs e retros

```
obsidian-vault/  
├─ CLAUDE.md           # contexto do projeto  
├─ _index.md           # índice gerado  
├─ 00-inbox.md         # capturas rápidas  
├─ daily/24-09-2026/  
│   ├─ session-10h40.md  
│   └─ transcript-*.md # fora do git  
├─ specs/  plans/  processes/  
├─ decisions/          # 0001-*.md ...  
├─ bugs/  retro/  
└─ .obsidian-memory.json
```

Os hooks cuidam do ciclo da sessão sozinhos



Os hooks só agem em projetos que têm vault. A memória começa quando você roda `init`, nunca antes.

O que o plugin faz por você

Retomada sem ferramentas

"Onde paramos?" é respondido direto do contexto injetado, sem ler arquivos.

Arquivo automático

Toda sessão é salva em `daily/`, com comandos e respostas, na compactação e no fim.

Decisões registradas

ADRs numerados e a seção "Do Not" evitam reabrir o que já foi decidido.

Git opcional e seguro

Commits locais só do vault, sem push. Verifica segredos antes de cada commit.

Obsidian opcional

Registra e abre o vault no Obsidian, com Daily Notes e Templates configurados.

Doctor e instalação

Confere Python, git, Obsidian e Git Bash. Instala o que falta, só com a sua aprovação.

Dois comandos dentro do Claude Code

```
# 1. adicionar o marketplace
/plugin marketplace add marcuswmc/obsidian-memory-plugin

# 2. instalar o plugin
/plugin install obsidian-memory@obsidian-memory

# 3. conferir os requisitos
/obsidian-memory:vault doctor
```

Os hooks vêm com o plugin: não é preciso editar o settings.json. Depois de instalar, reinicie o Claude Code ou rode /reload-plugins.

REQUISITO	MACOS / LINUX	WINDOWS
Claude Code	obrigatório	obrigatório
Python 3.8+	obrigatório	obrigatório
git	opcional	Git for Windows (Git Bash roda os hooks)
Obsidian	opcional	opcional

Comece com **init**, e o resto é automático

No seu projeto, rode `/obsidian-memory:vault init`. O Claude faz duas perguntas, ambas opcionais:

1. Versionar o vault com git?
Commits locais, só de `obsidian-vault/`. Se disser não num projeto com git, o vault vai para o `.gitignore`.

2. Usar o Obsidian?
Registra e abre o vault. Se não estiver instalado, oferece instalar.

Depois ele explora o projeto e preenche o `CLAUDE.md`.

COMANDO	O QUE FAZ
<code>vault init</code>	Cria o vault e pergunta sobre git e Obsidian
<code>vault save</code>	Resumo da sessão, atualiza o estado atual e comita
<code>vault status</code>	Estado do vault, git, Obsidian e alertas
<code>vault open</code>	Abre o vault no Obsidian
<code>vault doctor</code>	Confere e instala requisitos

Fale normalmente; o plugin entende

RETOMAR

"Onde paramos?"

Resposta imediata com o estado atual, o foco e os próximos passos, sem abrir nenhum arquivo.

REGISTRAR

"Registre esta decisão"

Cria `decisions/NNNN-*.md` e adiciona o item em Key Decisions e, se couber, em Do Not.

ENCERRAR

"Salve a sessão"

Escreve o resumo do dia, atualiza o Current State e comita o vault.

Idioma e formato

O plugin é em inglês. As notas são escritas no idioma da conversa, com datas no formato `dd-mm-yyyy`. As notas criadas pelo Claude seguem os templates: `spec`, `plan`, `process`, `decision`, `bug`, `retro` e `session`.

"Onde paramos?" com o plugin, em projetos reais

0

ferramentas usadas
nenhum arquivo aberto para responder

0

tokens de leitura adicionados
o contexto já chega pronto na sessão

~6 s

até a resposta completa
em uma única chamada ao modelo

PROJETO	CHAMADAS AO MODELO	FERRAMENTAS	CONTEXTO ANTES → DEPOIS DA RESPOSTA	TEMPO
Projeto de teste (teste-vault)	1	0	52.937 → 52.937 tokens	7,0 s
Este plugin (obsidian-memory-plugin)	1	0	80.667 → 80.667 tokens	5,7 s

Fonte: logs de uso do Claude Code (campo usage de cada chamada), sessões novas abertas com a pergunta "onde paramos?". O contexto inclui o prompt de sistema e as ferramentas configuradas em cada máquina; o que importa é que ele não cresce: a resposta sai do que o plugin já injetou.

Retomar um projeto: sem memória x com o plugin

-82%

tokens na janela de contexto para se orientar
~19.500 → ~3.500

-90%

tokens de entrada processados na retomada
~560 mil → ~57 mil

0

ferramentas para responder "onde paramos?"
contra ~8 leituras de arquivos

PARA SE ORIENTAR, O AGENTE PRECISA DE	SEM MEMÓRIA	COM OBSIDIAN MEMORY
Saber o que é o projeto e a stack	README + manifestos: ~2.200 tokens	
Entender o código e onde parou	Script principal, skill, CHANGELOG, git log: ~17.300 tokens	CLAUDE.md (~1.600) + último resumo (~400 a 900) + fim da conversa (até ~1.250): ~3.500 tokens , já no contexto
Saber o que já foi decidido e por quê	Não está no código: você explica de novo	

Base: tamanhos reais deste repositório (74 KB de código e docs; vault de 6,5 KB), considerando ~4 caracteres por token. Sem memória: ~8 leituras em ~9 chamadas, sobre uma base de ~53 mil tokens (prompt de sistema e ferramentas) que cresce a cada leitura. Os números variam com o tamanho do projeto: quanto maior o código, maior a economia.

Quanto isso soma em um mês

Por retomada de projeto, o plugin evita em média:

- ◆ **~16 mil tokens** a menos ocupando a janela de contexto, durante a sessão inteira
- ◆ **~500 mil tokens** de entrada a menos processados nas chamadas de orientação
- ◆ **~8 chamadas de ferramentas** e de 1 a 2 minutos de espera

Grande parte da entrada reprocessada vem do cache de prompt, que custa menos, mas conta para os limites de uso do seu plano.

RETOMADAS POR MÊS	JANELA DE CONTEXTO	ENTRADA PROCESSADA
10 (1 projeto, meio período)	~160 mil	~5 milhões
20 (1 projeto, todo dia)	~320 mil	~10 milhões
60 (3 projetos, todo dia)	~960 mil	~30 milhões

Fórmula: economia ≈ retomadas × (tokens de orientação evitados). Não entram na conta as retomadas depois de uma compactação nem o tempo gasto explicando o projeto de novo, e os dois aumentam a economia.

O contexto deixa de se perder

SEM O PLUGIN

- ◆ A compactação troca a conversa por um resumo curto
- ◆ Ao fechar a sessão, só fica o que está no código
- ◆ Motivos de decisões e próximos passos somem
- ◆ Outra sessão ou outro modelo começa do zero

COM O PLUGIN

- ◆ Antes de compactar, a conversa inteira é arquivada em `daily/`
- ◆ Depois da compactação, o `CLAUDE.md` volta ao contexto e o Claude registra o que foi feito
- ◆ Decisões viram ADRs, e a seção Do Not bloqueia retrocessos
- ◆ Qualquer modelo que leia o vault sabe onde o trabalho parou

A memória é do **projeto**, não da conversa: fica no disco, em Markdown, e o agente a consulta sempre que precisa.

Comparado às alternativas

	SEM MEMÓRIA	CLAUDE.MD ESCRITO À MÃO	OBSIDIAN MEMORY
Contexto do projeto no início	Não	Sim	Sim
Resumo da última sessão	Não	Não	Sim, injetado automaticamente
Conversas arquivadas	Não	Não	Sim, na compactação e no fim
Estado atual sempre atualizado	Não	Só se você lembrar	Sim, a cada save
Decisões com o porquê (ADRs)	Não	Manual	Sim, com templates
Histórico versionado	Não	Com o resto do código	Commits só do vault (opcional)
Navegação visual	Não	Não	Obsidian (opcional)

Garantias do plugin

Seu código fica intocado

Os commits incluem só `obsidian-vault/`. O que você deixou no stage não é mexido, e nada é enviado com push.

Segredos nunca entram no histórico

Antes de cada commit, o plugin procura chaves e tokens. Se achar algum, não comita e avisa na próxima sessão.

Nada acontece sem você pedir

Nenhum vault é criado sem `init`. Instalar programas ou reiniciar o Obsidian sempre pede a sua aprovação.

Conversas ficam fora do git

Transcrições e snapshots de memória ficam só no disco. O vault nunca é criado na pasta pessoal, em Downloads ou em Documentos.

Código aberto, local e sem dependência de serviço

LICENÇA MIT

Livre para usar, adaptar e redistribuir

Uso pessoal ou comercial. Todo o código está no GitHub para você ler antes de instalar.

100% LOCAL

Sua memória fica com você

Sem servidor, sem conta e sem telemetria. O vault é Markdown no seu disco, e o plugin não envia nada para lugar nenhum.

SEM DEPENDÊNCIAS

Simples de auditar

Um script em Python com a biblioteca padrão, um lançador bash e os hooks. Nada de pacotes externos.

Contribua

```
github.com/marcuswmc/  
obsidian-memory-plugin
```

- ◆ Abra uma issue com bugs ou ideias
- ◆ Envie um pull request com melhorias
- ◆ Adapte os templates ao seu jeito de trabalhar
- ◆ Deixe uma estrela para ajudar o projeto a crescer

COMECE AGORA

Retome qualquer projeto com uma pergunta.

```
/plugin marketplace add marcuswmc/obsidian-memory-plugin  
/plugin install obsidian-memory@obsidian-memory  
/obsidian-memory:vault init
```

Open source e gratuito · github.com/marcuswmc/obsidian-memory-plugin · Licença MIT